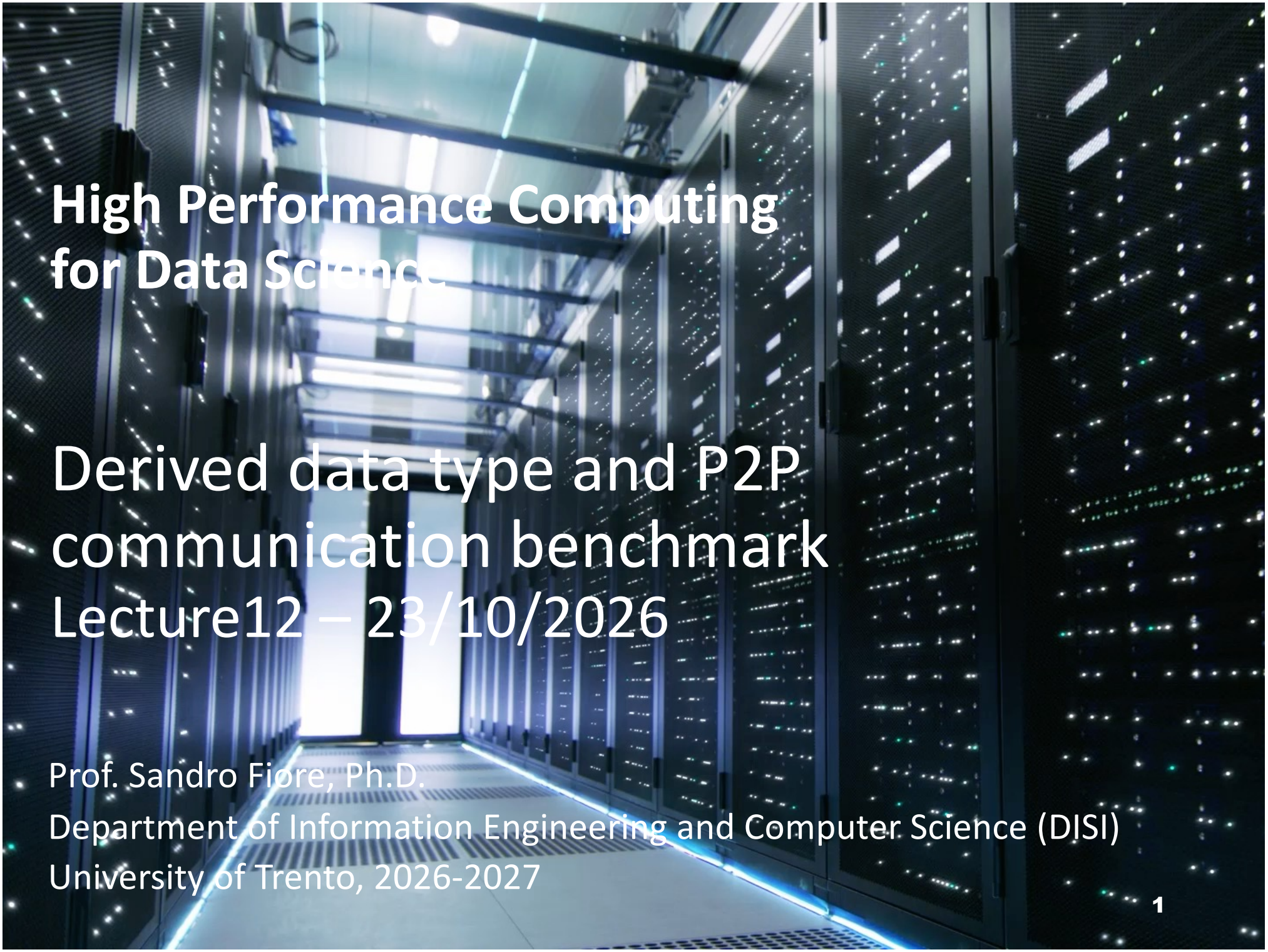


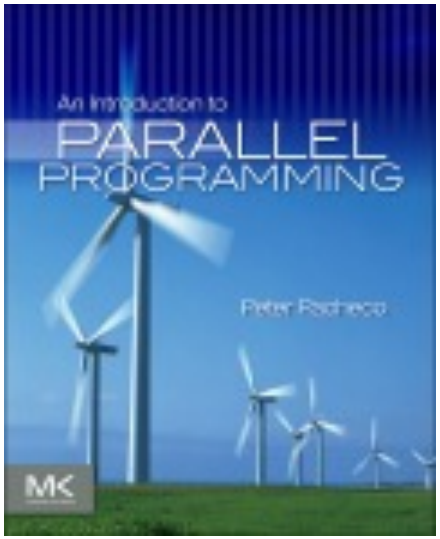
High Performance Computing for Data Science

Derived data type and P2P communication benchmark
Lecture12 – 23/10/2026

Prof. Sandro Fiore, Ph.D.
Department of Information Engineering and Computer Science (DISI)
University of Trento, 2026-2027

Today's outline and material

- Project topics
- Derived data type
- P2P communication benchmark



Today's material

Book
An Introduction to Parallel Programming by Peter Pacheco

Chapter 3
Distributed Memory Programming with MPI

Project topics

Parallel Bat Algorithm
Parallel solver per Aristotle's Number Puzzle
Parallel solver per Futoshiki puzzle (board at least 5 x 5, if possible 10 x 10)
Parallel Fast Hartley Transform
Parallel Bareiss algorithm
Parallel Levinson-Durbin algorithm
Parallel REQ (quantiles)
Parallel conjugate gradient
Parallel connected components
Parallel Q-Digest (quantiles)
Parallel Count-Min sketch
Parallel CPC (Compressed Probabilistic Counting) Sketch
Parallel Hyper Log Log Sketch
Parallel GSA (Gravitational Search Algorithm)
Parallel EM (Expectation-Maximization) clustering
Parallel CHARM (closed frequent itemsets mining)
Parallel Angle-Based Clustering
Parallel Selection
Parallel Spline Interpolation

Parallel Hough Transform
Parallel Discrete Haar transform
Parallel Sinkhorn factorization of a square matrix with positive entries
Parallel SVD factorization
Parallel Bareiss algorithm for computing the determinant of a matrix
Parallel Artificial Gorilla Troops Optimizer
Parallel Cholesky factorization of a dense, symmetric and positive definite square n x n matrix A
Parallel QR factorization
Parallel Power Method for eigenvalues
Parallel linear regression
Parallel SLASH
Parallel Matrix Inversion
Parallel CountSketch
Parallel tridiagonal linear system
Parallel numerical integration using Romberg's method
Parallel numerical integration using Gaussian Quadrature
Parallel Snyder Matrix Multiplication
Parallel Moth-Flame Optimization Algorithm
Parallel Raven Roosting Optimization Algorithm
Parallel root-finding of a polynomial

High Performance Computing for Data Science

Derived data types, timing and performance
Lecture 12 - 23/10/2026

MPI Derived Data Types

Prof. Sandro Fiore, Ph.D.
Department of Information Engineering and Computer Science (DISI)
University of Trento, 2026-2027

Derived datatypes

- Used to represent any collection of data items in memory by storing both the **types** of the items and **their relative locations** in memory.
- The idea is that if a function that sends data knows this information about a collection of data items, it can **collect the items from memory before they are sent**.
- Similarly, a function that receives data can distribute the items into their correct destinations in memory when they’re received.

Copyright © 2010, Elsevier Inc. All rights Reserved

Derived datatypes

- *Formally, consists of a sequence of basic MPI data types together with a displacement for each of the data types.*
- Trapezoidal Rule example:

Variable	Address
a	24
b	40
n	48

$$\{(\texttt{MPI_DOUBLE}, 0), (\texttt{MPI_DOUBLE}, 16), (\texttt{MPI_INT}, 24)\}$$

MORGAN KAUFMANN
Copyright © 2010, Elsevier Inc. All rights Reserved

MPI_Type create_struct

- Builds a derived datatype that consists of individual elements that have different basic types.
  - After its definition, a derived data type gets the status of “basic type”, so...

```
int MPI_Type_create_struct(
      int              count                          /* in  */,
      int              array_of_blocklengths[]        /* in  */,
      MPI_Aint         array_of_displacements[]       /* in  */,
      MPI_Datatype     array_of_types[]               /* in  */,
      MPI_Datatype*    new_type_p                     /* out */);
```

MORGAN KAUFMANN
Copyright © 2010, Elsevier Inc. All rights Reserved

MPI_Get_address

- Returns the address of the memory location referenced by location_p.
- The special type MPI_Aint is an integer type that is *big enough to store an address on the system*.

```
int MPI_Get_address(
      void*      location_p  /* in  */,
      MPI_Aint*  address_p   /* out */);
```

MORGAN KAUFMANN
Copyright © 2010, Elsevier Inc. All rights Reserved

MPI_Type_commit

- Allows the MPI implementation to optimize its internal representation of the datatype for use in communication functions.

```
int MPI_Type_commit(MPI_Datatype*  new_mpi_t_p  /* in/out */);
```

MORGAN KAUFMANN
Copyright © 2010, Elsevier Inc. All rights Reserved

MPI_Type_free

- When we're finished with our new type, this frees any additional storage used.

```
int MPI_Type_free(MPI_Datatype*  old_mpi_t_p  /* in/out */);
```

- Let's move to the get_input function of the trapezoidal rule (a,b,n).
- Do we need it? Ideas?
- Number of broadcast?

MK MORGAN KAUFMANN
Copyright © 2010, Elsevier Inc. All rights Reserved

Get_input that uses MPI_Bcast

```
void Get_input(
      int        my_rank  /* in  */,
      int        comm_sz  /* in  */,
      double*    a_p      /* out */,
      double*    b_p      /* out */,
      int*       n_p      /* out */) {

   if (my_rank == 0) {
      printf("Enter a, b, and n\n");
      scanf("%lf %lf %d", a_p, b_p, n_p);
   }
   MPI_Bcast(a_p, 1, MPI_DOUBLE, 0, MPI_COMM_WORLD);
   MPI_Bcast(b_p, 1, MPI_DOUBLE, 0, MPI_COMM_WORLD);
   MPI_Bcast(n_p, 1, MPI_INT, 0, MPI_COMM_WORLD);
}  /* Get_input */
```

MORGAN KAUFMANN

Copyright © 2010, Elsevier Inc. All rights Reserved

Get input function with a derived datatype (1)

```
void Get_input(int my_rank, int comm_sz, double* a_p, double* b_p,
      int* n_p) {
   MPI_Datatype input_mpi_t;

   Build_mpi_type(a_p, b_p, n_p, &input_mpi_t);

   if (my_rank == 0) {
      printf("Enter a, b, and n\n");
      scanf("%lf %lf %d", a_p, b_p, n_p);
   }
   MPI_Bcast(a_p, 1, input_mpi_t, 0, MPI_COMM_WORLD);

   MPI_Type_free(&input_mpi_t);
}  /* Get_input */
```

MORGAN KAUFMANN
Copyright © 2010, Elsevier Inc. All rights Reserved

Get input function with a derived datatype (2)

```
void Build_mpi_type(
      double*         a_p              /* in  */,
      double*         b_p              /* in  */,
      int*            n_p              /* in  */,
      MPI_Datatype*   input_mpi_t_p    /* out */) {

   int array_of_blocklengths[3] = {1, 1, 1};
   MPI_Datatype array_of_types[3] = {MPI_DOUBLE, MPI_DOUBLE, MPI_INT};
   MPI_Aint a_addr, b_addr, n_addr;
   MPI_Aint array_of_displacements[3] = {0};

   MPI_Get_address(a_p, &a_addr);
   MPI_Get_address(b_p, &b_addr);
   MPI_Get_address(n_p, &n_addr);
   array_of_displacements[1] = b_addr-a_addr;
   array_of_displacements[2] = n_addr-a_addr;
   MPI_Type_create_struct(3, array_of_blocklengths,
         array_of_displacements, array_of_types,
         input_mpi_t_p);
   MPI_Type_commit(input_mpi_t_p);
}  /* Build_mpi_type */
```

MORGAN KAUFMANN

Copyright © 2010, Elsevier Inc. All rights Reserved

Get input function with a derived datatype

```
void Get_input(int my_rank, int comm_sz, double* a_p, double* b_p,
      int* n_p) {
   MPI_Datatype input_mpi_t;

   Build_mpi_type(a_p, b_p, n_p, &input_mpi_t);

   if (my_rank == 0) {
      printf("Enter a, b, and n\n");
      scanf("%lf %lf %d", a_p, b_p, n_p);
   }
   MPI_Bcast(a_p, 1, input_mpi_t, 0, MPI_COMM_WORLD);

   MPI_Type_free(&input_mpi_t);
}  /* Get_input */
```

MORGAN KAUFMANN
Copyright © 2010, Elsevier Inc. All rights Reserved

High Performance Computing for Data Science

Derived data types, timing and performance
Lecture 12 - 23/10/2026

Taking timings

Prof. Sandro Fiore, Ph.D.
Department of Information Engineering and Computer Science (DISI)
University of Trento, 2026-2027

Elapsed parallel time

- Goal: Test/check, benchmarking
- Returns the number of seconds that have elapsed since some time in the past.
- **Wall clock time** (time elapsed between start & end execution of the code under testing) vs **CPU time** (clock call)

```
double MPI_Wtime(void);

        double start, finish;
        . . .
        start = MPI_Wtime();
        /* Code to be timed */
        . . .
        finish = MPI_Wtime();
        printf("Proc %d > Elapsed time = %e seconds\n"
               my_rank, finish-start);
```

https://www.mpich.org/static/docs/v3.3/www3/MPI_Wtime.html

MORGAN KAUFMANN

Copyright © 2010, Elsevier Inc. All rights Reserved

New MPI calls: MPI_Wtime

MPI_Wtime

Returns an elapsed time on the calling processor

Synopsis

```
double MPI_Wtime( void )
```

Return value

Time in seconds since an arbitrary time in the past.

Notes

This is intended to be a high-resolution, elapsed (or wall) clock. See `MPI_WTICK` to determine the resolution of `MPI_WTIME`. If the attribute `MPI_WTIME_IS_GLOBAL` is defined and true, then the value is synchronized across all processes in `MPI_COMM_WORLD`.

Notes for Fortran

This is a function, declared as `DOUBLE PRECISION MPI_WTIME()` in Fortran.

See Also also: MPI_Wtick, MPI_Comm_get_attr, MPI_Attr_get

Location:src/mpi/timer/wtime.c

https://www.mpich.org/static/docs/v3.1/www3/MPI_Wtime.html

Copyright © 2010, Elsevier Inc. All rights Reserved

gettimeofday

```
GETTIMEOFDAY(2)          FreeBSD System Calls Manual          GETTIMEOFDAY(2)

NAME
     gettimeofday, settimeofday -- get/set date and time

LIBRARY
     Standard C Library (libc, -lc)

SYNOPSIS
     #include <sys/time.h>

     int
     gettimeofday(struct timeval *tp, struct timezone *tzp);

     int
     settimeofday(const struct timeval *tp, const struct timezone *tzp);

DESCRIPTION
     The system's notion of the current Greenwich time and the current time
     zone is obtained with the gettimeofday() system call, and set with the
     settimeofday() system call.  The time is expressed in seconds and mi-
     croseconds since midnight (0 hour), January 1, 1970.  The resolution of
     the system clock is hardware dependent, and the time may be updated con-
     tinuously or in "ticks".  If tp or tzp is NULL, the associated time in-
     formation will not be returned or set.

     The structures pointed to by tp and tzp are defined in <sys/time.h> as:

     struct timeval {
             time_t          tv_sec;         /* seconds */
             suseconds_t     tv_usec;        /* and microseconds */
     };

     struct timezone {
             int     tz_minuteswest; /* minutes west of Greenwich */
             int     tz_dsttime;     /* type of dst correction */
     };

     The timezone structure indicates the local time zone (measured in minutes
     of time westward from Greenwich), and a flag that, if nonzero, indicates
     that Daylight Saving time applies locally during the appropriate part of
     the year.
```

MPI_Barrier

- Ensures that no process will return from calling it until every process in the communicator has started calling it.

```
int MPI_Barrier(MPI_Comm    comm    /* in */);
```

- Why are we talking about MPI_Barrier here?



MORGAN KAUFMANN

Copyright © 2010, Elsevier Inc. All rights Reserved

MPI_Barrier

```
double local_start, local_finish, local_elapsed, elapsed;
. . .
MPI_Barrier(comm);
local_start = MPI_Wtime();
/* Code to be timed */
. . .
local_finish = MPI_Wtime();
local_elapsed = local_finish - local_start;
MPI_Reduce(&local_elapsed, &elapsed, 1, MPI_DOUBLE,
    MPI_MAX, 0, comm);

if (my_rank == 0)
    printf("Elapsed time = %e seconds\n", elapsed);
```

MORGAN KAUFMANN
Copyright © 2010, Elsevier Inc. All rights Reserved

High Performance Computing for Data Science

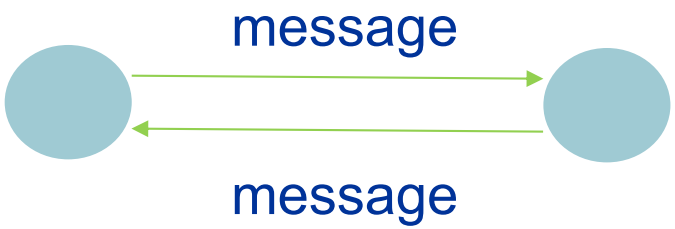
P2P Communication Benchmark
Lecture12 - 23/10/2026

Point-to-point communication benchmark

Prof. Sandro Fiore, Ph.D.
Department of Information Engineering and Computer Science (DISI)
University of Trento, 2026-2027

Benchmarking p2p communication

Write a program to measure the time it takes to send 1, 2, 4, ..., 1M, ... bytes (arrays) from one processor to another using ***MPI_Send*** and ***MPI_Recv***.



New concepts:

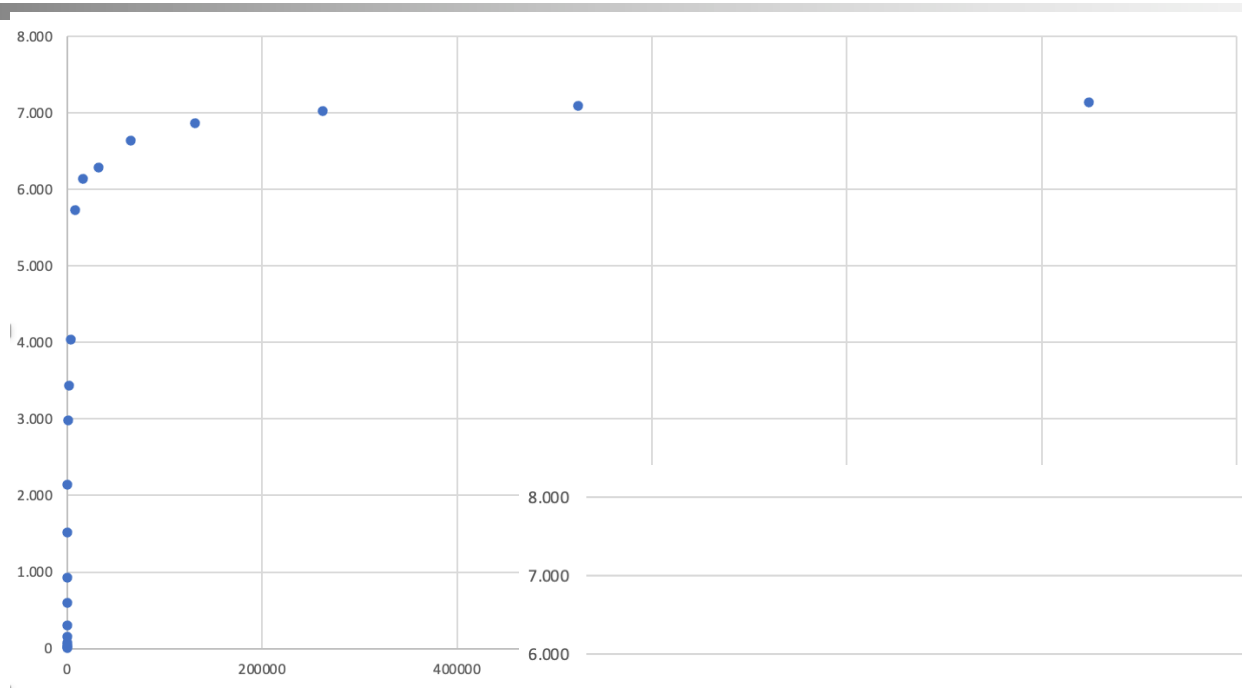
- *PBS "placing" strategies (class)*
- *MPI_Wtime (class)*

Practice with *malloc/free, MPI_Send & MPI_Receive*

Used to estimate the bandwidth

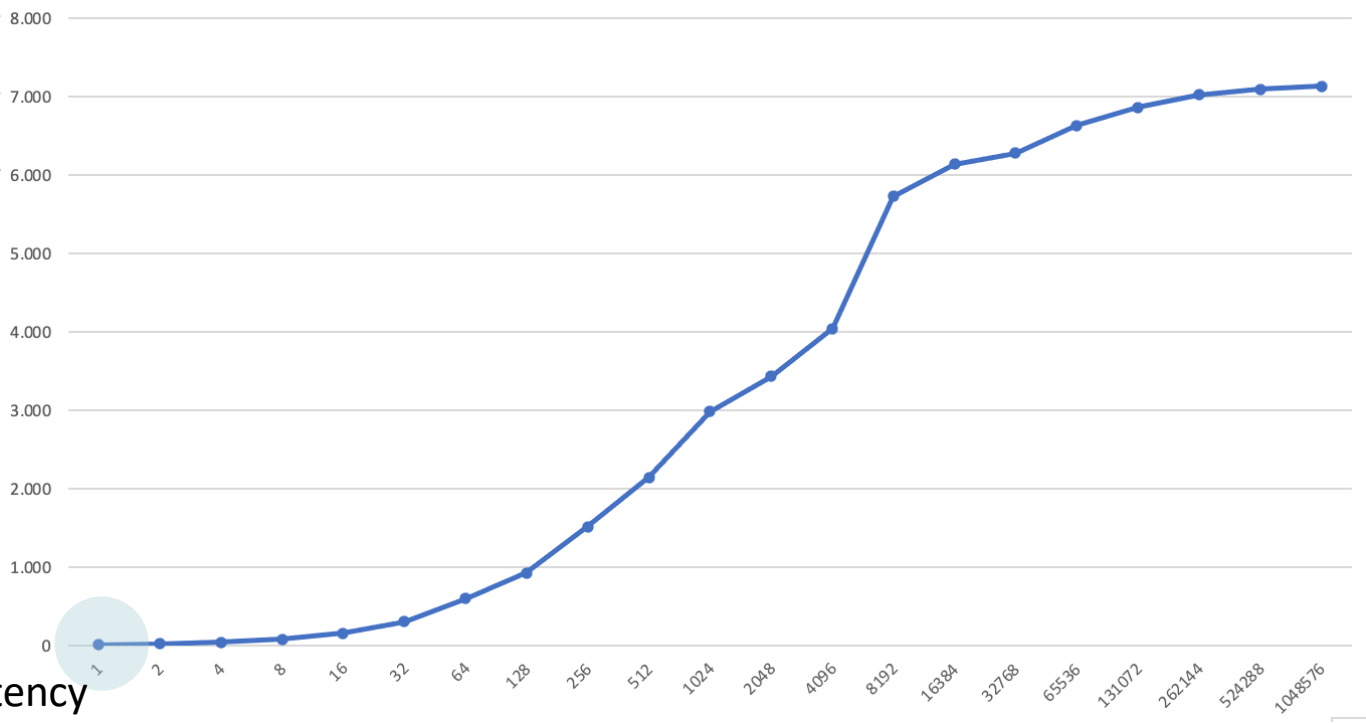
MORGAN KAUFMANN
Copyright © 2010, Elsevier Inc. All rights Reserved

Performance: charts



Linear scale

Log scale



Latency

MORGAN KAUFMANN

Copyright © 2010, Elsevier Inc. All rights Reserved

Expected output

- A **2-process MPI** application
- **Print** the size, time, and rate in MB/sec for each message length
- **Plot** the **MB/sec** in a chart (message size on the X axis, use log scale)

```
n       time (sec)      Rate (MB/sec)
1       0.0             6
2       0.0             1
4       0.0             2
8       0.0             5
16      0.0             1
32      0.0             2
64      0.0             4
128     0.0             6
256     0.0             1
512     0.0             1
1024    0.0             1
2048    0.0             1
4096    0.0             1
8192    0.0             2
16384   0.0             3
32768   0.0             2
65536   0.0             2
131072  0.0             2
262144  0.0             2
524288  0.0             2
1048576 0.0             2
```

MORGAN KAUFMANN

Copyright © 2010, Elsevier Inc. All rights Reserved

Run over multiple configurations

```
#PBS -l select=2:ncpus=1:mem=2gb -l place=pack
#PBS -l select=2:ncpus=1:mem=2gb -l place=scatter
#PBS -l select=2:ncpus=1:mem=2gb -l place=pack:excl
#PBS -l select=2:ncpus=1:mem=2gb -l place=scatter:excl
```

Important for your project

MORGAN KAUFMANN
Copyright © 2010, Elsevier Inc. All rights Reserved

Points to address in the benchmark

- How to take timing
- Max message size (experiment boundaries)
- Bandwidth calculation (formula)
- Statistical approach to get an accurate estimate
- Placing strategies (which one to use)
- “Execution time” mathematical model
  - f(n) where n is message size
- Design and implementation of the parallel app
- Benchmark run
- Results reporting

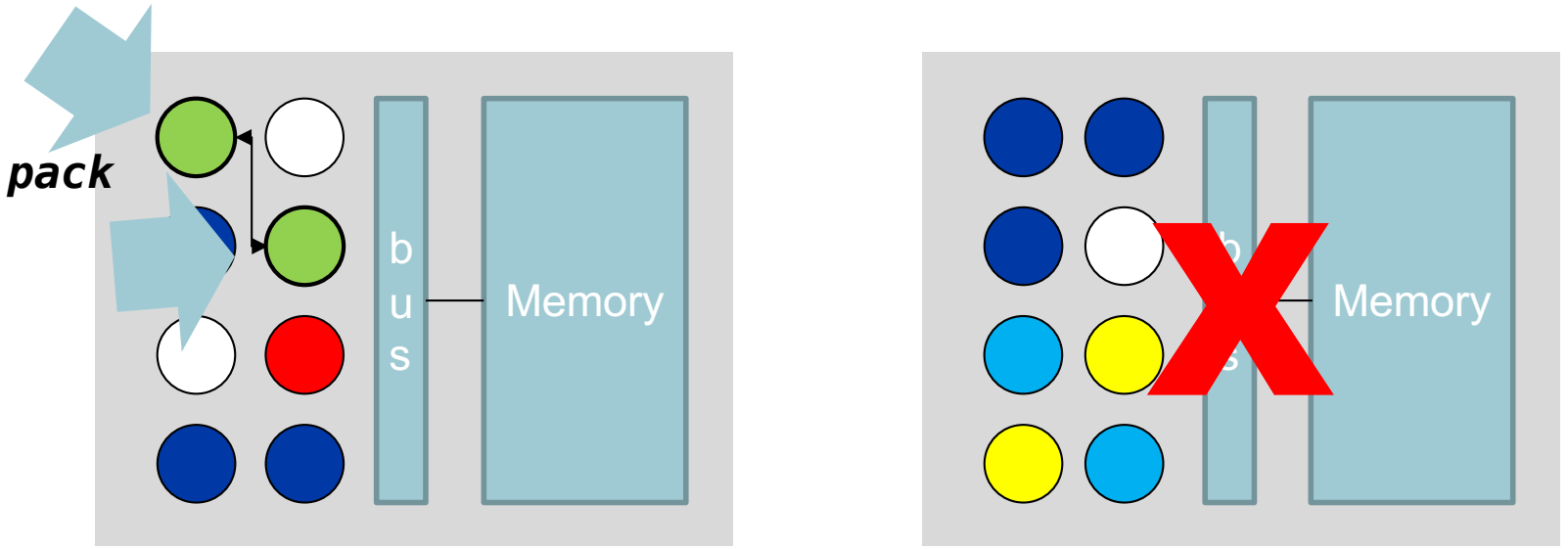
Copyright © 2010, Elsevier Inc. All rights Reserved

Backup slides

Copyright © 2010, Elsevier Inc. All rights Reserved

PBS Placing strategies (1)

1. #PBS –l select=2:ncpus=1:mem=2gb –l *place*=***pack***



Free core

Cores allocated to my app

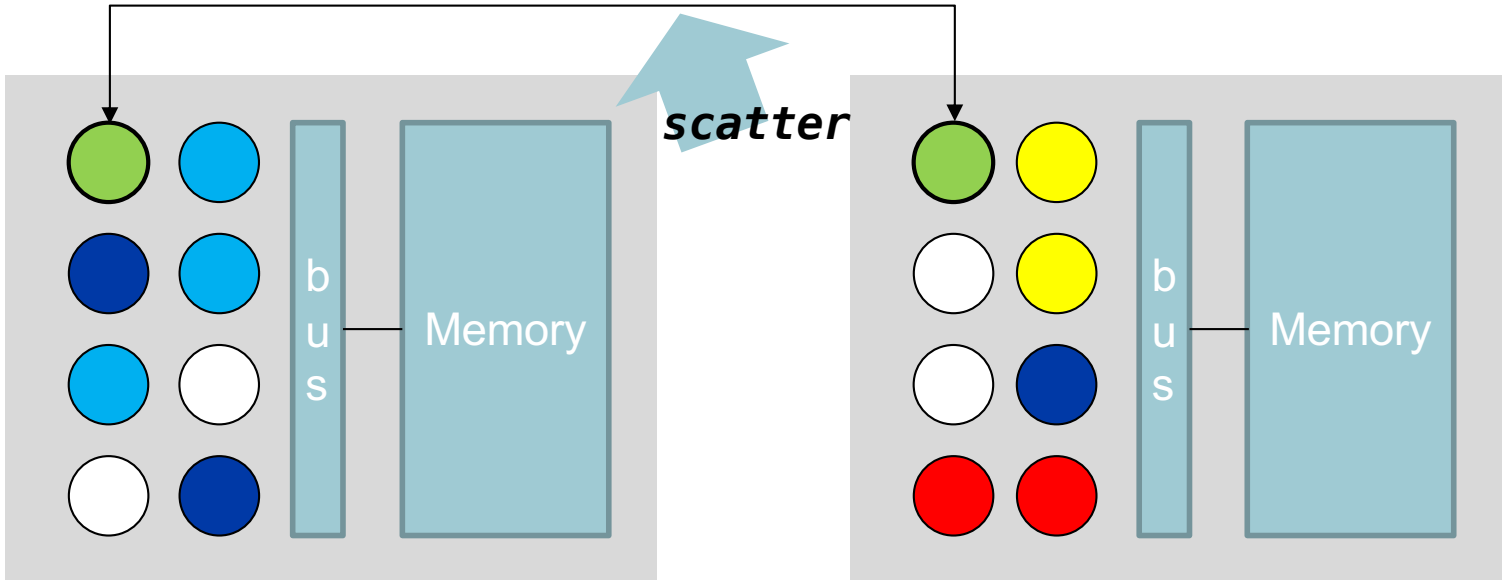
Cores allocated to other users' apps

2 chunks (select 2) by 1 core (ncpus 1) on the same node (pack)

MORGAN KAUFMANN
Copyright © 2010, Elsevier Inc. All rights Reserved

PBS Placing strategies (2)

2. `#PBS -l select=2:ncpus=1:mem=2gb -l place=scatter`



Free core

Cores allocated to my app

Cores allocated to other users' apps

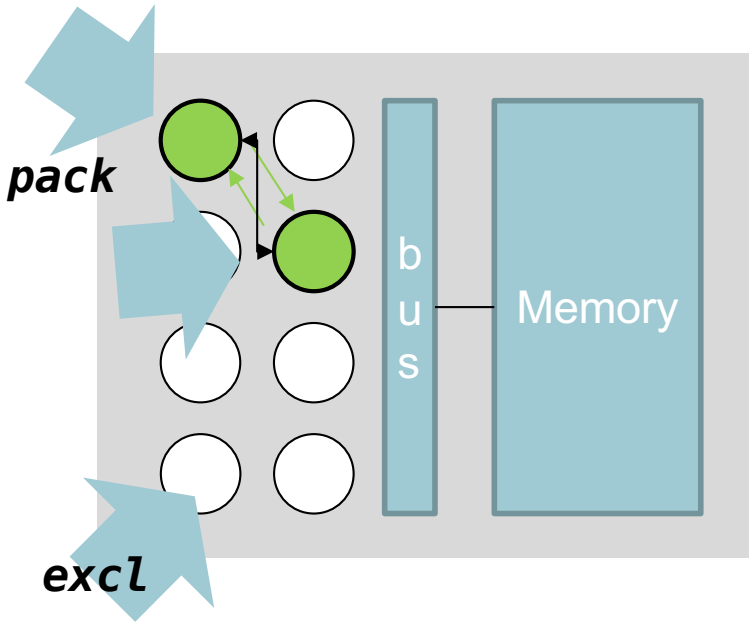
2 chunks (select 2) by 1 core (ncpus 1) on two different nodes (scatter)

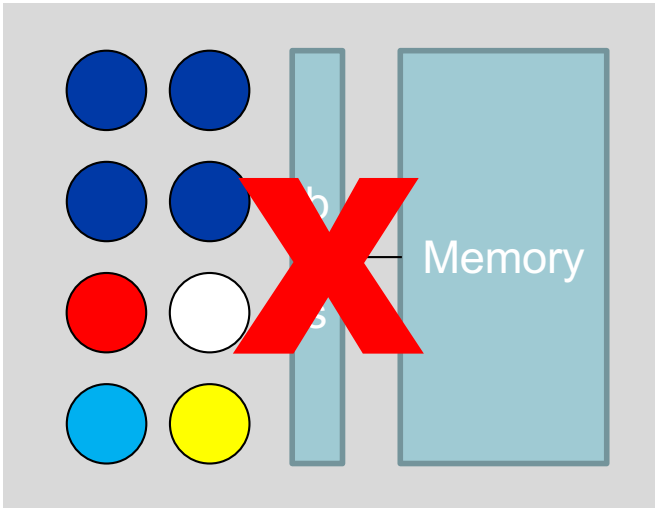
MORGAN KAUFMANN

Copyright © 2010, Elsevier Inc. All rights Reserved

PBS Placing strategies (3)

3. `#PBS -l select=2:ncpus=1:mem=2gb -l place=`***`pack:excl`***





2 chunks (select 2) by 1 core (ncpus 1) on the same node (pack) with exclusive allocation (excl)

Free core

Cores allocated to my app

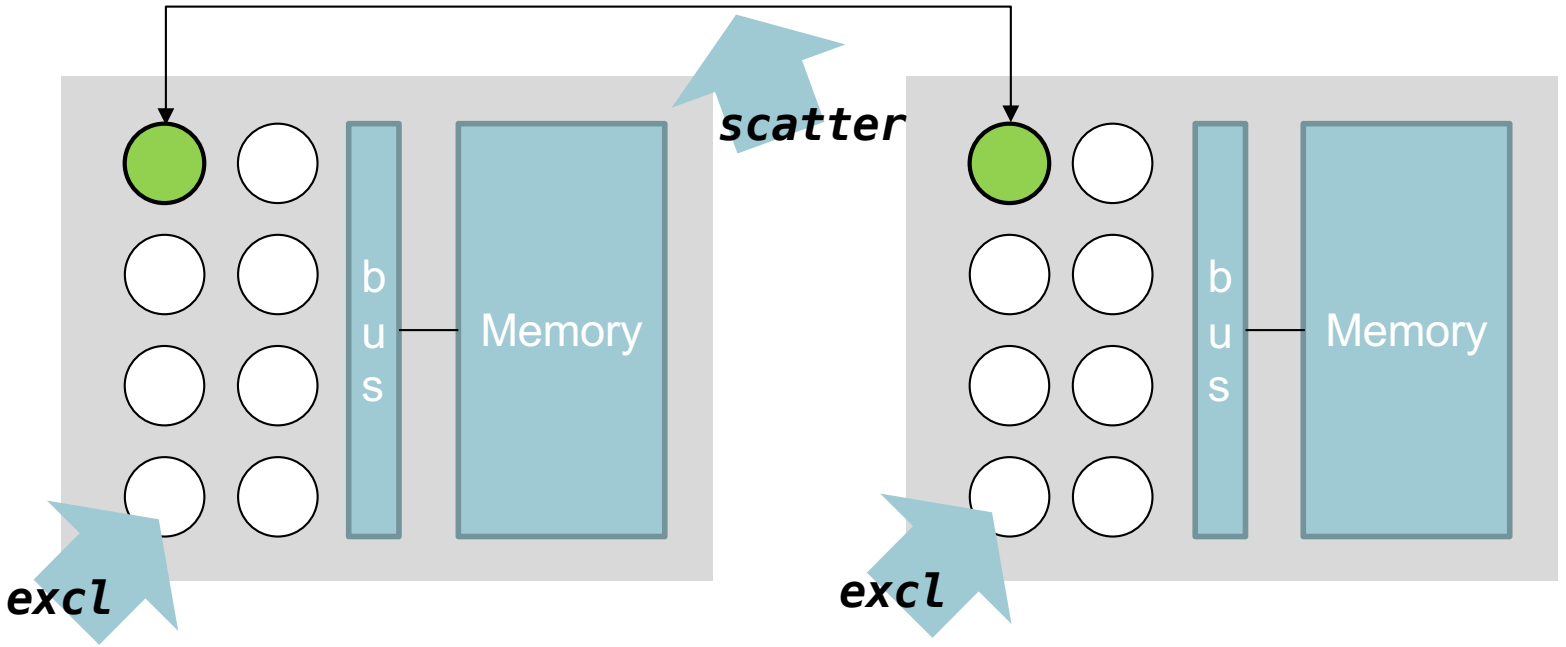
Cores allocated to other users' apps

MORGAN KAUFMANN

Copyright © 2010, Elsevier Inc. All rights Reserved

PBS Placing strategies (4)

4. `#PBS -l select=2:ncpus=1:mem=2gb -l place=`***`scatter:excl`***



Free core

Cores allocated to my app

2 chunks (select 2) by 1 core (ncpus 1) on two different nodes (scatter) with exclusive allocation (excl)

MORGAN KAUFMANN

Copyright © 2010, Elsevier Inc. All rights Reserved

Parallel job submission

```
$ more test3.sh

#!/bin/bash

#PBS -l select=2:ncpus=1:mem=2gb

# set max execution time
#PBS -l walltime=0:00:30

# set the queue
#PBS -q shortCPUQ

module load MPICH/3.4.2-GCC-10.3.0

mpiexec -n 2 ./mpi_wrapper.sh ./p2p
```



MORGAN KAUFMANN

Copyright © 2010, Elsevier Inc. All rights Reserved

mpi_wrapper.sh

```
#!/bin/bash

IFACE=$(ip -4 route get 192.168.108.1 |
        awk '{for (i=1;i<=NF;i++) if ($i=="dev") {print $(i+1);
exit}}')

if [[ -z "$IFACE" || "$IFACE" == "lo" ]]; then
    echo "ERROR: No valid Ethernet interface found on $(hostname)"
>&2
    exit 1
fi

export UCX_TLS=tcp,self,sm
export UCX_NET_DEVICES="$IFACE"

echo "$(hostname): UCX_NET_DEVICES=$UCX_NET_DEVICES"

exec "$@"
```

MORGAN KAUFMANN
Copyright © 2010, Elsevier Inc. All rights Reserved

Feedback form

- Throughout the entire HPC4DS course there will be a form available for your feedback
  - Please provide any comment about:
    - Pros
    - Cons
    - Aspects that were not clear enough during the class
    - Any other feedback you think can be relevant for the course
    - …
- Information are gathered in an anonymous way

https://docs.google.com/forms/d/e/1FAIpQLScSVpNPfMP8poNNz-nLl9xlKlHkeBM2x_JgWyA0qQu6mOBl5A/viewform